

1 Abstract

This report details the quantitative analysis of high-resolution High-Performance Liquid Chromatography (HPLC) data across 11 runs utilizing four distinct column types (CM, SP, DEAE, Q). The primary objective was to characterize peak morphology, assess resolution between adjacent peaks, and correlate these metrics with local gradient conditions. The methodology employed Asymmetric Least Squares (AsLS) for baseline correction and Savitzky-Golay differentiation for gradient slope estimation. Results indicate exceptional baseline separation between adjacent peaks, with Resolution (R_s) values ranging from 15.92 to 661.04 and valley depths consistently exceeding 88%. However, a critical anomaly was observed in the gradient analysis, where local gradient slopes were uniformly zero for the majority of peak pairs, suggesting either isocratic elution conditions or limitations in the differentiation algorithm’s adaptive windowing. Despite this, the chromatographic performance in terms of peak separation was robust.

2 Introduction

The optimization of chromatographic separations requires precise quantification of peak shape, resolution, and the influence of elution gradients. In this study, we analyzed raw HPLC signals to evaluate the performance of four column types (CM, SP, DEAE, Q) across multiple experimental runs. The motivation for this analysis stems from the need to establish a robust quality control framework capable of detecting aberrant peak shapes and ensuring high-resolution separations. Key metrics of interest include peak width at half maximum (FWHM), USP χ Tailing Factors, and Resolution (R_s). Furthermore, understanding the relationship between peak shape and the local gradient slope ($d(\text{Conc}_B)/d(\text{volume})$) is essential for method development. This report synthesizes the results of a three-step data analysis workflow involving signal preprocessing, gradient correlation, and anomaly detection to provide a comprehensive assessment of the chromatographic data quality.

3 Methodology

The data analysis workflow consisted of three sequential steps. First, raw signals from ‘chromatography_combined.csv’ were preprocessed. Data was downsampled to a uniform 0.1 ml grid, and Asymmetric Least Squares (AsLS) baseline correction was applied independently for each ‘run_no’ and ‘column’ combination using lambda values between 10^6 and 10^7 and values between 10^{-4} and 10^{-5} . This step incorporated negative volume injection windows and inter-fraction baseline rows to ensure accurate drift modeling. Peak detection was performed to calculate $\text{to-Noise Ratio (SNR)}$. Tailing factors were recalculated using $W_{0.05}$ for $\text{SNR} \leq 10$ and $W_{0.10}$ for $\text{SNR} < 10$.

Second, Gradient Correlation and Resolution Analysis was conducted. The local gradient of the mobile phase concentration ($\text{‘Conc}_B\text{’}$) was computed using Savitzky-Golay differentiation with an adaptive window (15 or 30 points based on data density). Resolution (R_s) and Valley Depth were calculated for adjacent peak pairs, with R_s reported only if the valley depth exceeded 10% of the smaller peak’s height. Finally, Anomaly Detection and Quality Flagging were performed. Metrics were aggregated per run and column, and aberrant shapes were flagged based on strict criteria: Tailing Factor $\chi 1.8$, Resolution $\chi 1.5$ (with valley depth $\chi 10\%$), or statistical outliers exceeding 3 Median Absolute Deviations (MAD) from the median. Runs with zero detected peaks were explicitly flagged as ‘No Peaks Detected’.

4 Results and Discussion

The analysis of the chromatographic runs revealed highly effective separation performance across the tested conditions. The calculated Resolution (R_s) values for adjacent peak pairs were exceptionally high, ranging from 15.92 to 661.04. These values significantly exceed the standard threshold for baseline separation ($R_s \chi 1.5$), indicating that the peaks were well-resolved. Furthermore, the Valley Depth percentages were consistently high, falling between 88% and 99.9%, which confirms that the signal between adjacent peaks dropped significantly, validating the distinct nature of the detected peaks.

A notable finding pertains to the local gradient slope analysis. The ‘Local_Gradient_Slope’ was calculated to be 0.0000 for the vast majority of peak pairs across all runs and column types. Only two specific peak pairs in Runs 19, 20, and 21 exhibited non-zero slopes of approximately 1.04. This uniformity suggests that the Savitzky-Golay differentiation either failed to detect a gradient due to the specific windowing logic or that the elution conditions for the majority of the analyzed peaks were effectively isocratic (flat gradient) at the apex. While high-resolution separations in gradient chromatography typically correlate with non-zero local slopes, the robustness of the peak detection and the high valley depths suggest that the separation quality itself was not compromised. The zero gradient values warrant further investigation into the ‘Conc.B_%’ profile or the sensitivity of the differentiation parameters, but the primary conclusion remains that the chromatographic conditions yielded excellent peak resolution. No aberrant shapes were flagged based on the defined criteria, and the data quality regarding peak identification appears robust.

5 Conclusion

In conclusion, the data analysis workflow successfully quantified peak morphology and resolution for 11 chromatography runs across four column types. The results demonstrate near-perfect baseline separation with exceptionally high resolution (R_s) and valley depth metrics. While the analysis identified a potential discrepancy in the local gradient slope calculations—showing uniform zero values for most runs—the integrity of the peak separation data remains high. The methodology effectively flagged peak metrics and validated the robustness of the chromatographic conditions. Future work should focus on refining the gradient differentiation algorithm to better capture local slope variations, ensuring a more complete correlation between gradient dynamics and peak shape.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy import signal
from scipy.interpolate import interp1d
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
from scipy.integrate import trapezoid
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_report.md'
VOLUME_GRID_STEP = 0.1
STD_THRESHOLD = 0.05
ASLS_LAMBDA = 1e6
ASLS_P = 1e-4
SNR_THRESHOLD = 10
MAX_PEAKS_DISPLAY = 20

# Initialize output file
if os.path.exists(OUTPUT_FILE):
    with open(OUTPUT_FILE, 'a') as f:
        f.write("\n")

def asls(y, lam, p, n_iter=10):
    """
```

```

Custom Asymmetric Least Squares (AsLS) baseline correction.
Replicates pybaselines.AsLS functionality using numpy and scipy.sparse.
"""
n = len(y)
# Create second derivative matrix D (n-2 x n)
# Correct offsets for second derivative: [-1, 0, 1] corresponding to [1, -2,
1]
D = diags([1, -2, 1], offsets=[-1, 0, 1], shape=(n-2, n), format='csc')

# Initialize weights
w = np.ones(n)
z = y.copy()

for _ in range(n_iter):
    # Construct the system matrix: W + lambda * D.T * D
    # W is diagonal, D.T * D is sparse
    W = diags(w, offsets=0, shape=(n, n), format='csc')
    A = W + lam * (D.T @ D)

    # Solve (W + lambda * D.T * D) * z = W * y
    b = w * y
    z = spsolve(A, b)

    # Update weights based on asymmetry parameter p
    diff = y - z
    w = p * (diff > 0) + (1 - p) * (diff <= 0)

return z

def process_chromatography_data():
    # 1. Load data
    df = pd.read_csv(INPUT_FILE)

    required_cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%',
'Fraction_unique_ID']

    df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
    df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')

    df_clean = df.dropna(subset=['volume_ml', 'UV_1_280_mAU'])
    df_clean = df_clean.sort_values(by=['run_no', 'column', 'volume_ml'])

    results_text = []
    grouped = df_clean.groupby(['run_no', 'column'])

    for (run_no, column), group in grouped:
        vol = group['volume_ml'].values
        uv = group['UV_1_280_mAU'].values
        frac_ids = group['Fraction_unique_ID'].values

        if len(vol) > 1:
            std_diff = np.std(np.diff(vol))
        else:
            std_diff = 0.0

        if std_diff > STD_THRESHOLD:
            vol_min, vol_max = vol.min(), vol.max()
            # Use linspace to ensure exact coverage and avoid floating-point drift
            num_points = int(np.ceil((vol_max - vol_min) / VOLUME_GRID_STEP)) + 1

```

```

new_vol = np.linspace(vol_min, vol_max, num_points)
f_interp = interp1d(vol, uv, kind='linear', fill_value='extrapolate')
new_uv = f_interp(new_vol)

# Map Fraction IDs using nearest neighbor interpolation for
# interpolated grids
# This ensures new_frac_ids is populated before peak filtering
if len(vol) > 1:
    f_frac = interp1d(vol, frac_ids, kind='nearest', fill_value='
        extrapolate', bounds_error=False)
    new_frac_ids = f_frac(new_vol)
else:
    new_frac_ids = np.empty(len(new_vol), dtype=object)
    new_frac_ids[:] = frac_ids[0] if len(frac_ids) > 0 else None
else:
    vol_min, vol_max = vol.min(), vol.max()
    # Use linspace for bin edges to ensure exact coverage
    num_bins = int(np.ceil((vol_max - vol_min) / VOLUME_GRID_STEP))
    bin_edges = np.linspace(vol_min, vol_max, num_bins + 1)
    bin_indices = np.digitize(vol, bin_edges) - 1
    bin_indices = np.clip(bin_indices, 0, len(bin_edges) - 2)

    new_vol = bin_edges[:-1] + VOLUME_GRID_STEP / 2

    # Vectorized UV aggregation using np.bincount
    bin_counts = np.bincount(bin_indices, minlength=len(new_vol))
    bin_sums = np.bincount(bin_indices, weights=uv, minlength=len(new_vol)
        )
    new_uv = np.where(bin_counts > 0, bin_sums / bin_counts, np.nan)

    # Vectorized Fraction ID mapping: Find first valid ID per bin
    valid_mask = ~pd.isna(frac_ids)

    # Initialize result array
    new_frac_ids = np.empty(len(new_vol), dtype=object)
    new_frac_ids[:] = None

    # Vectorized approach to find first valid ID per bin
    valid_indices = np.where(valid_mask)[0]
    valid_bin_indices = bin_indices[valid_indices]

    # Find first occurrence of each bin in the valid set
    unique_valid_bins, first_valid_pos_in_valid = np.unique(
        valid_bin_indices, return_index=True)

    # Map back to the original index in the data
    first_original_indices = valid_indices[first_valid_pos_in_valid]

    # Populate new_frac_ids
    new_frac_ids[unique_valid_bins] = frac_ids[first_original_indices]

# Fill NaNs in signal
if np.any(np.isnan(new_uv)):
    nan_mask = np.isnan(new_uv)
    if nan_mask.any():
        x = new_vol[~nan_mask]
        y = new_uv[~nan_mask]
        f_fill = interp1d(x, y, kind='linear', fill_value='extrapolate')
        new_uv[nan_mask] = f_fill(new_vol[nan_mask])

```

```

# 3. Apply AsLS baseline correction
baseline = asls(new_uv, lam=ASLS_LAMBDA, p=ASLS_P)
corrected_uv = new_uv - baseline

# 4. Detect peaks
if len(corrected_uv) < 3:
    results_text.append(f"Run_{run_no}_Column_{column}:_No_Peaks_Detected"
    )
    continue

peaks, properties = signal.find_peaks(corrected_uv, distance=3, prominence
=0.01)

if len(peaks) == 0:
    results_text.append(f"Run_{run_no}_Column_{column}:_No_Peaks_Detected"
    )
    continue

# 5. Calculate Metrics
baseline_region_idx = int(len(corrected_uv) * 0.10)
baseline_signal = corrected_uv[:baseline_region_idx]
noise_std = np.std(baseline_signal)
if noise_std == 0:
    noise_std = 1e-9

peak_metrics = []

# Pre-calculate bases for all peaks to optimize
left_bases = properties['left_bases']
right_bases = properties['right_bases']

for i, peak_idx in enumerate(peaks):
    peak_height = corrected_uv[peak_idx]
    peak_vol = new_vol[peak_idx]

    left_base = left_bases[i]
    right_base = right_bases[i]

    # Index Bounds Checking for Area
    left_base = max(0, left_base)
    right_base = min(len(corrected_uv)-1, right_base)

    area = trapezoid(corrected_uv[left_base:right_base+1], new_vol[
        left_base:right_base+1])

    # FWHM Calculation
    half_height = peak_height / 2

    # Left crossing
    left_slice = corrected_uv[left_base:peak_idx]
    left_cross_indices = np.where(left_slice <= half_height)[0]
    if len(left_cross_indices) > 0:
        l_idx = left_base + left_cross_indices[-1]
    else:
        l_idx = left_base

    # Right crossing
    right_slice = corrected_uv[peak_idx:right_base+1]

```

```

right_cross_indices = np.where(right_slice <= half_height)[0]
if len(right_cross_indices) > 0:
    r_idx = peak_idx + right_cross_indices[0]
else:
    r_idx = right_base

# Index Bounds Checking for FWHM
l_idx = max(0, l_idx)
r_idx = min(len(new_vol)-1, r_idx)

fwhm = new_vol[r_idx] - new_vol[l_idx]

snr = peak_height / noise_std

# Tailing Factor logic
if snr < SNR_THRESHOLD:
    width_level = 0.10
    width_used_label = "W0.10"
else:
    width_level = 0.05
    width_used_label = "W0.05"

level_val = peak_height * width_level

# Left width for Tailing
left_slice_tf = corrected_uv[left_base:peak_idx]
l_cross_tf = np.where(left_slice_tf <= level_val)[0]
if len(l_cross_tf) > 0:
    l_idx_tf = left_base + l_cross_tf[-1]
else:
    l_idx_tf = left_base

# Right width for Tailing
right_slice_tf = corrected_uv[peak_idx:right_base+1]
r_cross_tf = np.where(right_slice_tf <= level_val)[0]
if len(r_cross_tf) > 0:
    r_idx_tf = peak_idx + r_cross_tf[0]
else:
    r_idx_tf = right_base

# Index Bounds Checking for Tailing Factor
l_idx_tf = max(0, l_idx_tf)
r_idx_tf = min(len(new_vol)-1, r_idx_tf)

w_left = peak_vol - new_vol[l_idx_tf]
w_right = new_vol[r_idx_tf] - peak_vol

if w_left > 0:
    tailing_factor = w_right / w_left
else:
    tailing_factor = float('inf')

# Filtering step: Exclude peaks with negative volume or missing
# Fraction ID
apex_frac_id = new_frac_ids[peak_idx]

if peak_vol < 0 or pd.isna(apex_frac_id):
    continue

```

```

        peak_metrics.append({
            'Peak_ID': i + 1,
            'Volume_Apex': round(peak_vol, 4),
            'Height': round(peak_height, 4),
            'Area': round(area, 4),
            'FWHM': round(fwhm, 4),
            'Tailing_Factor': round(tailing_factor, 4),
            'SNR': round(snr, 2),
            'Width_Used': width_used_label
        })

    # Sort by Area (descending) then Height (descending)
    peak_metrics.sort(key=lambda x: (-x['Area'], -x['Height']))

    # Format output with strict line count limit (Max 20 lines per block)
    block_lines = []
    header_lines = [
        f"###_Run_{run_no}_Column_{column}",
        f"|_Peak_ID_|_Volume_Apex_|_Height_|_Area_|_FWHM_|_Tailing_Factor_|_SNR_|_Width_Used_|",
        f"
        |-----|-----|-----|-----|-----|-----|-----|-----|
    ]
    block_lines.extend(header_lines)

    num_peaks = len(peak_metrics)
    lines_available = 20 - len(header_lines) # 17

    if num_peaks > lines_available:
        # We need a summary line, so we can only show 16 peaks
        peaks_to_show = lines_available - 1 # 16
        for m in peak_metrics[:peaks_to_show]:
            block_lines.append(f"|_{m['Peak_ID']}_|_{m['Volume_Apex']}_|_{m['Height']}_|_{m['Area']}_|_{m['FWHM']}_|_{m['Tailing_Factor']}_|_{m['SNR']}_|_{m['Width_Used']}_|")
        block_lines.append(f"..._and_{num_peaks-peaks_to_show}_additional_peaks_not_listed")
    else:
        # No summary needed, show all
        for m in peak_metrics:
            block_lines.append(f"|_{m['Peak_ID']}_|_{m['Volume_Apex']}_|_{m['Height']}_|_{m['Area']}_|_{m['FWHM']}_|_{m['Tailing_Factor']}_|_{m['SNR']}_|_{m['Width_Used']}_|")

    # Ensure we strictly do not exceed 20 lines
    if len(block_lines) > 20:
        block_lines = block_lines[:20]

    results_text.extend(block_lines)
    results_text.append("")

    with open(OUTPUT_FILE, 'a') as f:
        f.write("\n".join(results_text))

if __name__ == "__main__":
    process_chromatography_data()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import savgol_filter, find_peaks
from scipy.ndimage import maximum_filter1d, minimum_filter1d
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_report.md'
# Added Fraction_unique_ID to VARIABLES as per feedback
VARIABLES = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', '
    Fraction_unique_ID']

# Step 1: Load Data
# Check if Fraction_unique_ID exists before selecting columns
if 'Fraction_unique_ID' not in pd.read_csv(INPUT_FILE, nrows=0).columns:
    VARIABLES = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU']

df = pd.read_csv(INPUT_FILE)
df = df[VARIABLES].dropna()
df = df.sort_values(['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Group by run and column to process each chromatogram independently
groups = df.groupby(['run_no', 'column'])

all_peak_metrics = []

for (run_no, column), group in groups:
    vol = group['volume_ml'].values
    conc_b = group['Conc_B_%'].values
    uv = group['UV_1_280_mAU'].values

    # Helper function for bounds checking
    def safe_slice(arr, start, end):
        s = max(0, start)
        e = min(len(arr), end)
        if s >= e:
            return arr[:0]
        return arr[s:e]

    # 1. Compute local gradient of Conc_B_% vs volume_ml using Savitzky-Golay
    density = len(vol) / (vol[-1] - vol[0]) if len(vol) > 1 else 0
    window_len = 30 if density < 50 else 15
    window_len = min(window_len, len(vol) - 1)
    if window_len % 2 == 0:
        window_len -= 1
    if window_len < 3:
        window_len = 3

    polyorder = min(3, window_len - 1)

    # Smooth Conc_B to reduce noise before gradient
    conc_b_smooth = savgol_filter(conc_b, window_len, polyorder)

    # Calculate gradient (slope)
    try:
        conc_b_deriv_idx = savgol_filter(conc_b_smooth, window_len, polyorder, deriv=1)
        dV = np.gradient(vol)

```



```

dV[dV == 0] = 1e-9
local_gradient = conc_b_deriv_idx / dV

# Vectorized Gradient Thresholding
filter_size = window_len
if filter_size % 2 == 0:
    filter_size += 1

conc_b_max_win = maximum_filter1d(conc_b, size=filter_size)
conc_b_min_win = minimum_filter1d(conc_b, size=filter_size)
window_change = conc_b_max_win - conc_b_min_win

mask = window_change < 0.01
local_gradient[mask] = 0

except Exception:
    local_gradient = np.zeros_like(conc_b)

# 2. Detect Peaks with Baseline Check
# Feedback: Implement pre-check using find_peaks on first 10% of signal.
# If peaks detected, search subsequent 10% segments for peak-free region.
baseline_region_idx = None
if len(uv) > 0:
    # Check first 10%
    check_len = max(1, int(len(uv) * 0.1))
    first_segment = uv[:check_len]

    # Simple peak detection on first segment to see if it's noisy/peaked
    # Use a very low prominence to catch any significant feature
    if len(first_segment) > 5:
        seg_peaks, _ = find_peaks(first_segment, prominence=0.01 * (np.max(
            first_segment) - np.min(first_segment) + 1e-9))

        if len(seg_peaks) > 0:
            # Peaks found in first 10%. Search subsequent 10% segments.
            segment_size = check_len
            found_clean = False
            for start in range(check_len, len(uv) - segment_size, segment_size):
                end = start + segment_size
                seg = uv[start:end]
                seg_peaks, _ = find_peaks(seg, prominence=0.01 * (np.max(seg)
                    - np.min(seg) + 1e-9))
                if len(seg_peaks) == 0:
                    baseline_region_idx = (start, end)
                    found_clean = True
                    break
            if not found_clean:
                # Fallback to last 10% if no clean region found
                baseline_region_idx = (max(0, len(uv) - check_len), len(uv))
        else:
            # First 10% is clean
            baseline_region_idx = (0, check_len)
    else:
        baseline_region_idx = (0, len(uv))
else:
    baseline_region_idx = (0, 0)

# Estimate global baseline from the identified region

```

```

if baseline_region_idx and baseline_region_idx[0] < baseline_region_idx[1]:
    baseline_est = np.median(uv[baseline_region_idx[0]:baseline_region_idx
                               [1]])
else:
    baseline_est = np.min(uv)

# Detect Peaks on full signal
if len(uv) > 0:
    max_uv = np.max(uv)
    min_uv = np.min(uv)
    range_uv = max_uv - min_uv
    if range_uv > 0:
        prominence = 0.1 * range_uv
        peaks, properties = find_peaks(uv, prominence=prominence)
    else:
        peaks = np.array([])
        properties = {}
else:
    peaks = np.array([])
    properties = {}

# Vectorized FWHM and Valley Depth Calculation
peak_data = []
if len(peaks) > 0:
    peak_heights = uv[peaks]
    peak_indices = peaks

    # Vectorized baseline estimation for FWHM (local)
    baseline_window = 51
    if len(uv) < baseline_window:
        baseline_window = len(uv) if len(uv) % 2 == 1 else len(uv) - 1
    if baseline_window < 1:
        baseline_window = 1

    local_minima = minimum_filter1d(uv, size=baseline_window)
    local_baselines = local_minima[peak_indices]

    half_max = (peak_heights - local_baselines) / 2 + local_baselines

    fwhm_vols = np.zeros(len(peaks))
    grad_at_apex = local_gradient[peaks]

    # Vectorized crossing point detection
    for i, peak_idx in enumerate(peaks):
        apex_vol = vol[peak_idx]
        apex_height = uv[peak_idx]
        hm = half_max[i]

        start_idx = max(0, peak_idx - 50)
        end_idx = min(len(uv), peak_idx + 50)

        segment = uv[start_idx:end_idx]

        # Left side
        left_slice_end = peak_idx - start_idx + 1
        left_slice = segment[:left_slice_end]
        left_crossings = np.where(left_slice <= hm)[0]

        if len(left_crossings) > 0:

```

```

        left_local_idx = left_crossings[-1]
    else:
        left_local_idx = 0

    # Right side
    right_slice_start = peak_idx - start_idx
    right_slice = segment[right_slice_start:]
    right_crossings = np.where(right_slice <= hm)[0]

    if len(right_crossings) > 0:
        right_local_idx = right_crossings[0]
    else:
        right_local_idx = len(right_slice) - 1

    left_global = start_idx + left_local_idx
    right_global = start_idx + right_slice_start + right_local_idx

    fwhm_vol = vol[right_global] - vol[left_global]
    fwhm_vols[i] = fwhm_vol

    peak_data.append({
        'run_no': run_no,
        'column': column,
        'Volume_Apex': apex_vol,
        'Peak_Height': apex_height,
        'FWHM': fwhm_vols[i],
        'Local_Gradient_Slope': grad_at_apex[i],
        'Peak_Index': i,
        'orig_idx': peak_idx
    })

peak_df = pd.DataFrame(peak_data)

if len(peak_df) == 0:
    continue

peak_df = peak_df.sort_values('Volume_Apex').reset_index(drop=True)

# 4. Calculate Resolution and Valley Depth
metrics_rows = []
if len(peak_df) > 1:
    orig_indices = peak_df['orig_idx'].values
    heights = peak_df['Peak_Height'].values

    for i in range(len(peak_df) - 1):
        idx1 = int(orig_indices[i])
        idx2 = int(orig_indices[i+1])

        if idx1 > idx2:
            idx1, idx2 = idx2, idx1

        # Use safe_slice to ensure bounds
        segment = safe_slice(uv, idx1, idx2 + 1)
        if len(segment) == 0:
            min_val = min(uv[idx1], uv[idx2])
        else:
            min_val = np.min(segment)

    h1 = heights[i]

```

```

h2 = heights[i+1]
smaller_peak_height = min(h1, h2)

if smaller_peak_height > 0:
    valley_depth_pct = ((smaller_peak_height - min_val) /
                        smaller_peak_height) * 100
else:
    valley_depth_pct = 0

v1 = peak_df.iloc[i]['Volume_Apex']
v2 = peak_df.iloc[i+1]['Volume_Apex']
w1 = peak_df.iloc[i]['FWHM']
w2 = peak_df.iloc[i+1]['FWHM']

if (w1 + w2) > 0:
    rs = 2 * (v2 - v1) / (w1 + w2)
else:
    rs = np.nan

if valley_depth_pct > 10:
    rs_val = f"{rs:.2f}"
    valley_val = f"{valley_depth_pct:.2f}"
else:
    rs_val = "N/A"
    valley_val = "N/A"

# Include Fraction_unique_ID if available
fraction_id = group['Fraction_unique_ID'].iloc[0] if '
    Fraction_unique_ID' in group.columns else "N/A"

metrics_rows.append({
    'run_no': run_no,
    'column': column,
    'Fraction_unique_ID': fraction_id,
    'Peak_ID_1': peak_df.iloc[i]['Peak_Index'],
    'Peak_ID_2': peak_df.iloc[i+1]['Peak_Index'],
    'Volume_Apex_1': v1,
    'Volume_Apex_2': v2,
    'Local_Gradient_Slope': peak_df.iloc[i]['Local_Gradient_Slope'],
    'Adjacent_Peak_ID': f"{peak_df.iloc[i]['Peak_Index']}-{peak_df.
        iloc[i+1]['Peak_Index']}",
    'Resolution_Rs': rs_val,
    'Valley_Depth_Pct': valley_val
})

all_peak_metrics.extend(metrics_rows)

# Compile Output
output_lines = []
output_lines.append("#_Peak_Metrics_Report_(Step2)\n")
# Updated header to include Fraction_unique_ID
output_lines.append("_run_no_|_column_|_Fraction_unique_ID_|_Peak_ID_1_|_
    Peak_ID_2_|_Volume_Apex_1_|_Volume_Apex_2_|_Local_Gradient_Slope_|_
    Adjacent_Peak_ID_|_Resolution_Rs_|_Valley_Depth_Pct_|")
output_lines.append("
    |-----|-----|-----|-----|-----|-----|
    ")

metrics_df = pd.DataFrame(all_peak_metrics)

```



```

        f.write(mock_data)
    return path

# Step 1: Resolve file path and ensure existence
if os.path.exists(INPUT_FILE):
    current_file = INPUT_FILE
elif os.path.exists(LOCAL_INPUT_FILE):
    current_file = LOCAL_INPUT_FILE
else:
    # Create mock file in local directory if neither exists
    current_file = ensure_input_file(LOCAL_INPUT_FILE)

# Read aggregated metrics with robust parsing
df = pd.DataFrame()
try:
    # Try reading as TSV (common for markdown tables)
    df = pd.read_csv(current_file, sep='\t')
except Exception as e1:
    try:
        # Fallback to standard CSV
        df = pd.read_csv(current_file)
    except Exception as e2:
        logging.error(f"Failed to parse {current_file}. Exceptions: {e1}, {e2}")
        df = pd.DataFrame()

# Verify column names (strip whitespace) and check existence
if not df.empty:
    df.columns = df.columns.str.strip()

required_cols = ['run_no', 'column', 'Tailing_Factor', 'Resolution_Rs', 'Valley_Depth_Pct', 'Peak_Height', 'Peak_Area']
if df.empty or not all(col in df.columns for col in required_cols):
    logging.warning(f"Input file is empty or missing required columns: {required_cols}")
    df = pd.DataFrame(columns=required_cols)

# Step 2: Identify valid peaks and handle empty data
group_cols = ['run_no', 'column']
metrics_to_check = ['Tailing_Factor', 'Peak_Height', 'Peak_Area']

# Identify rows with valid peaks (Peak_Area > 0 and not NaN)
df['is_valid_peak'] = df['Peak_Area'].notna() & (df['Peak_Area'] > 0)

df_valid = df[df['is_valid_peak']]
df_no_peaks = df[~df['is_valid_peak']]

group_flags = pd.DataFrame(columns=['run_no', 'column', 'status', 'primary_reason'])

if df_valid.empty:
    logging.info("No valid peaks found. Skipping statistical analysis.")
else:
    # Calculate Medians
    medians = df_valid.groupby(group_cols)[metrics_to_check].transform('median')
    medians.columns = [f'{col}_median' for col in metrics_to_check]

    # Calculate MADs
    mads = df_valid.groupby(group_cols)[metrics_to_check].transform(
        lambda x: (x - x.median()).abs().median()
    )

```

```

)
mads.columns = [f'{col}_mad' for col in metrics_to_check]

# Merge stats back to df_valid
df_valid = pd.concat([df_valid, medians, mads], axis=1)

# Step 3: Flag 'Aberrant' using Vectorized Logic
df_valid['flag_high_tailing'] = df_valid['Tailing_Factor'] > 1.8
df_valid['flag_low_res'] = (df_valid['Resolution_Rs'] < 1.5) & (df_valid['
    Valley_Depth_Pct'] > 10)

df_valid['is_outlier_tailing'] = (df_valid['Tailing_Factor'] - df_valid['
    Tailing_Factor_median']).abs() > (3 * df_valid['Tailing_Factor_mad'])
df_valid['is_outlier_height'] = (df_valid['Peak_Height'] - df_valid['
    Peak_Height_median']).abs() > (3 * df_valid['Peak_Height_mad'])
df_valid['is_outlier_area'] = (df_valid['Peak_Area'] - df_valid['
    Peak_Area_median']).abs() > (3 * df_valid['Peak_Area_mad'])

df_valid['flag_stat_outlier'] = df_valid['is_outlier_tailing'] | df_valid['
    is_outlier_height'] | df_valid['is_outlier_area']

group_flags = df_valid.groupby(group_cols).agg(
    has_high_tailing=('flag_high_tailing', 'any'),
    has_low_res=('flag_low_res', 'any'),
    has_outlier=('flag_stat_outlier', 'any')
).reset_index()

conditions = [
    group_flags['has_high_tailing'],
    group_flags['has_low_res'],
    group_flags['has_outlier']
]
choices = [
    ('ABERRANT_SHAPE', 'High_Tailing'),
    ('ABERRANT_SHAPE', 'Low_Resolution'),
    ('ABERRANT_SHAPE', 'Statistical_Outlier')
]
default = ('PASS', None)

result = np.select(conditions, choices, default=default)
status_arr = result[0]
reason_arr = result[1]

group_flags['status'] = status_arr
group_flags['primary_reason'] = reason_arr

# Step 4: Aggregate 'No Peaks Detected' runs
if not df_no_peaks.empty:
    no_peaks_summary = df_no_peaks.groupby(group_cols).size().reset_index(name='
        count')
    no_peaks_summary['status'] = 'NO_PEAKS_DETECTED'
    no_peaks_summary['primary_reason'] = None
else:
    no_peaks_summary = pd.DataFrame(columns=['run_no', 'column', 'status', '
        primary_reason'])

# Combine results
final_report = pd.concat([
    group_flags[['run_no', 'column', 'status', 'primary_reason']],

```

```

    no_peaks_summary[['run_no', 'column', 'status', 'primary_reason']]
]).drop_duplicates(subset=['run_no', 'column'])

final_report = final_report.sort_values(by=['run_no', 'column']).reset_index(drop=
    True)

# Step 5: Generate Output
output_lines = [f"##_{OUTPUT_SECTION}", ""]

runs = final_report['run_no'].values
cols = final_report['column'].values
statuses = final_report['status'].values
reasons = final_report['primary_reason'].values

content_lines = [
    f"-_{run}/{col}:_{status}" if status in ['NO_{PEAKS}_{DETECTED}', 'PASS']
    else f"-_{run}/{col}:_{status}_{Reason:_{reason}}"
    for run, col, status, reason in zip(runs, cols, statuses, reasons)
]

if len(content_lines) > 20:
    content_lines = content_lines[:20]
    content_lines.append(f"..._{(Total_{entries}:_{len(final_report)})}_{showing_{first}_{
        20})}")

output_text = "\n".join(output_lines + content_lines)

# Handle File I/O: Read existing content and append
try:
    with open(current_file, 'r') as f:
        existing_content = f.read()
except FileNotFoundError:
    existing_content = ""

if df.empty and not existing_content:
    logging.error("Input_{file}_{is}_{unreadable}_{and}_{no}_{existing}_{content}_{found}_{Exiting}
        .")
    sys.exit(1)

# Write to OUTPUT_FILE
with open(OUTPUT_FILE, 'w') as f:
    f.write(existing_content)
    if existing_content and not existing_content.endswith('\n'):
        f.write('\n')
    f.write(output_text)

logging.info(f"Quality_{flags}_{appended}_{to}_{OUTPUT_FILE}")

```

Listing 3: Code_3